Bank SOC Assistant: Local LLM-Based Security Alert Analyzer

1. Project Overview

This project demonstrates how a local Large Language Model can assist a bank Security Operations Center in analyzing security alerts. The system receives a bank security alert, performs rule-based pre-analysis, sends the alert to a local Ollama model, maps the result to MITRE ATT&CK, and generates an incident report with cryptographic integrity protection.

The project uses only free and local AI. It does not use OpenAI API or any cloud LLM provider.

2. Problem Statement

Bank SOC teams receive many alerts from online banking systems, ATM systems, login systems, firewalls, email gateways, and endpoint security tools. Analysts must understand each alert quickly, identify indicators of compromise, estimate business impact, and recommend remediation steps.

Manual alert triage can be slow, repetitive, and inconsistent. A local LLM can help summarize alerts and produce structured analysis while keeping sensitive bank data inside the local environment.

3. Objectives

The main objectives are:

- Build a Streamlit interface for SOC alert analysis.
- Use Ollama local models such as llama3.2, mistral, and qwen2.5.
- Perform deterministic rule-based pre-analysis before the LLM.
- Extract suspicious IP addresses, URLs, emails, and hashes.
- Ask the local LLM to return structured SOC analysis in JSON.
- Map common attack types to MITRE ATT&CK references.
- Generate JSON and TXT incident reports.
- Protect report integrity using SHA-256 and HMAC signatures.
- Provide a simple dashboard for analyzed alerts.
- Add privacy-conscious IOC enrichment.
- Compare rule confidence with LLM confidence.
- Add analyst decision actions and report downloads.
- Add a presentation mode for classroom demonstrations.

4. System Architecture

The application is divided into small Python modules:

- app.py: Streamlit user interface and workflow controller.
- ollama_client.py: Sends alerts to the local Ollama API.
- rule_engine.py: Detects suspicious patterns and extracts indicators.
- mitre_mapper.py: Maps attack categories to MITRE ATT&CK.
- report_generator.py: Saves incident reports in JSON and TXT.
- crypto_utils.py: Generates SHA-256 hashes and HMAC signatures.
- sample_alerts.json: Contains realistic bank SOC demo alerts.
- threat_intel.py: Performs local IOC enrichment and rule confidence scoring.

5. Local LLM Integration

The system calls Ollama locally at:

```
http://localhost:11434/api/chat
```

Supported model choices in the interface are:

- llama3.2
- mistral
- qwen2.5
- custom Ollama model name

The LLM prompt instructs the model to act as a senior bank SOC analyst and return only JSON.

6. Rule-Based Pre-Analysis

Before the LLM is called, the rule engine checks for:

- SQL injection keywords such as UNION SELECT and OR '1'='1'
- XSS payloads such as <script> and javascript:
- Phishing indicators such as urgent password reset messages
- Brute-force patterns such as repeated failed logins
- Malware indicators such as suspicious PowerShell or trojan activity
- Data exfiltration indicators such as large outbound uploads
- ATM fraud patterns such as suspicious withdrawals

The rule engine also extracts indicators of compromise:

- IP addresses
- URLs

- email addresses
- MD5, SHA-1, and SHA-256 hashes

7. IOC Enrichment

The application enriches extracted indicators locally. It assigns heuristic risk scores to IP addresses, URLs, emails, and hashes. It also provides manual lookup links for public sources such as VirusTotal, Talos, and urlscan.io.

The full bank alert is not sent to online services. This design keeps sensitive alert content private while still giving analysts useful investigation links.

8. Structured LLM Output

The LLM is required to return JSON with these fields:

- alert_summary
- attack_type
- severity
- confidence_score
- indicators_of_compromise
- affected_assets
- business_impact_for_bank
- technical_explanation
- recommended_remediation_steps
- false_positive_possibility
- final_incident_report

The application validates this response and handles invalid JSON or missing fields.

9. MITRE ATT&CK Mapping

The project includes a simple MITRE mapper:

- Brute force maps to T1110 Brute Force
- Phishing maps to T1566 Phishing
- SQL injection and web exploit map to T1190 Exploit Public-Facing Application
- Malware maps to an Execution / Malware category
- Data exfiltration maps to the Exfiltration tactic

This mapping helps connect the alert analysis to a recognized cybersecurity framework.

10. Analyst Workflow

The app includes workflow controls that allow an analyst to:

- confirm an incident
- mark an alert as a false positive
- escalate to Tier 2
- close an alert

It also displays a generated incident timeline showing alert receipt, rule analysis, local LLM analysis, containment guidance, and report integrity.

11. Report Generation

After each analysis, the system saves:

- A JSON incident report
- A TXT incident report
- An integrity metadata file

Reports are stored in the reports/ directory.

Each report includes:

- original alert text
- selected model
- rule-based findings
- IOC enrichment
- confidence comparison
- LLM analysis
- MITRE mapping
- incident timeline
- analyst decision
- final incident report

12. Cryptographic Integrity

The project uses two integrity mechanisms:

- SHA-256 hash: detects if report content changes.
- HMAC-SHA256 signature: proves the hash was signed using the project secret key.

The verification feature allows the user to select a saved report and check whether it has been modified.

13. Dashboard

The dashboard shows:

- number of analyzed alerts
- number of stored reports
- last generated report hash
- alerts by severity
- alerts by attack type

This provides a small SOC management view for the demo.

14. Demo Scenarios

The project includes sample bank alerts for:

- online banking brute-force login
- phishing email targeting bank employees
- SQL injection attempt on banking portal
- suspicious ATM withdrawals
- abnormal outbound traffic and data exfiltration
- malware alert on an employee workstation
- impossible travel employee login
- suspicious SWIFT transfer approval
- ransomware behavior on a file server
- privileged account misuse

15. Privacy Benefits

Local LLMs are useful for a bank SOC because alerts can contain sensitive information such as customer identifiers, infrastructure names, IP addresses, employee emails, and transaction patterns.

By using Ollama locally, the project avoids sending sensitive alert data to an external cloud AI provider.

16. Limitations

- The project is an educational prototype.
- LLM analysis can be wrong and must be reviewed by a human analyst.
- MITRE mapping is simple and rule-based.
- The application does not connect to a real SIEM.
- HMAC key management is simplified for demonstration.

- Local model quality depends on the installed Ollama model.

17. Conclusion

The Bank SOC Assistant shows how local LLMs can support security alert triage in privacy-sensitive environments. It combines deterministic rule checks with LLM-generated analysis, MITRE ATT&CK mapping, report generation, and integrity verification. This makes it suitable for a university cybersecurity presentation about AI-assisted SOC workflows.